

Digital Signal Processing and Infrasound Analysis of the 2022 Hunga Tonga Eruption

Introduction

The Hunga Tonga–Hunga Ha'apai volcanic eruption in January 2022 generated atmospheric waves that were recorded by stations around the world. In this project, digital signal processing (DSP) techniques are used to analyze the recorded signals.

The project includes implementing convolution and the discrete-time Fourier transform (DTFT), applying three FIR filters to 201 recorded time series, and examining the propagation of the atmospheric waves. Finally, the arrival times of the high-frequency infrasound waves are used to estimate their propagation celerity.

Task 1

```
In [1]: # These code snippets that parse parts of run_provided.m  
# have kindly been shared by Markus Bjørklund  
#  
# Please note that you'll need to do some work to put these snippets  
# into work in the appropriate order, for example, in a Jupyter notebook  
# or by calling things in the right order from a main()  
  
#For parsing and general analysis  
import numpy as np  
import matplotlib.pyplot as plt  
import h5py  
import os  
  
#For datetime objects  
from matplotlib import dates  
from datetime import datetime, timedelta
```

```

import dateutil.parser

#For map
import cartopy.crs as ccrs
import cartopy.feature as cfeature
from geopy.distance import great_circle

# Set path for data
path = os.path.abspath('.') + '/' #Working directory path (Where program is run from)
fn_list = os.listdir(path + '/data') #List of all file names

def parse(num_files=None):
    #Read files and store contents. Accepts user input for number of files to read.
    if num_files:
        N_files = num_files
    else:
        N_files = len(fn_list)

    # Initialize
    lats = np.zeros(N_files)
    lons = np.zeros(N_files)
    dt = np.zeros(N_files)

    N_samples = 720000 #Size of the datasets in the files. Hard coded unless you want dynamic allocation (lists, sl
    data_collection = np.zeros((N_files,N_samples)) # all station data
    times_collection = np.zeros((N_files,N_samples), dtype=datetime) # all corresponding time vectors

    #Loop over files
    for ii in range(N_files):
        if (ii%50) == 0:
            print('Loading station ', ii, ' out of ', N_files-1)

        this_fn = h5py.File(path + 'data/' + fn_list[ii], 'r') # The file path we will read

        # Read the latitude and longitude attributes for the station and put into vectors
        lats[ii] = this_fn.attrs['latitude']
        lons[ii] = this_fn.attrs['longitude']

        #Read tree structure and load dataset into aggregate array
        level1 = list(this_fn.keys()) # /waveforms

```

```

level2 = list(this_fn[level1[0]].keys()) # /waveforms/dataset_name
dataset = this_fn['/' + level1[0] + '/' + level2[0]] #Reading dataset in subgroup
data = dataset[:] #Extract numpy array from dataset object

#If missing data, pad with zeros
if len(data) < N_samples:
    #data = np.pad(data, (0, N_samples-len(data)), 'constant', constant_values=np.nan)
    data = np.pad(data, (0, N_samples-len(data)), 'constant', constant_values=0)

data_collection[ii,:] = data #Loading numpy array into aggregate data array

# Figure out the start time and generate a time vector
start_time_str = dataset.attrs['starttime']
start_time = dateutil.parser.isoparse(start_time_str) #Time attribute on ISO-format

dt[ii] = dataset.attrs['delta']
deltas = np.linspace(0, dt[ii]*(N_samples-1), N_samples) #A duration which can be added to a datetime object
times = start_time + deltas*timedelta(seconds = 1)
times_collection[ii,:] = times #Loading times into aggregate array

print('done')
return data_collection, times_collection, lats, lons, dt

def plot_map(lats, lons, tonga_latlon):
    northward_offset = 90 #As in sample program
    central_lat = tonga_latlon[0] + northward_offset
    central_lon = tonga_latlon[1]

    fig = plt.figure(figsize=(15, 15))
    ax = fig.add_subplot(1, 1, 1,
        projection=ccrs.AzimuthalEquidistant(central_latitude = central_lat, central_longitude=central_lon))

    #Stock background (Natural Earth)
    ax.stock_img()

    #Make the map global rather than have it zoom in to the extents of any plotted data
    ax.set_global()

    #Make gridlines
    ax.gridlines(crs=ccrs.PlateCarree())

```

```

#Plot stations and Hunga Tonga
ax.scatter(tonga_latlon[1], tonga_latlon[0], marker = 'x', linewidth = 2,
color='red', transform=ccrs.PlateCarree()) #Transform is PlateCarree regardless of projection

ax.scatter(lons, lats, marker = '^', linewidth = 0.5, facecolor='none',
edgecolor='magenta', transform=ccrs.PlateCarree()) #Transform is PlateCarree regardless of projection

plt.show()

def circle_distance(n_files, lats, lons, tonga_latlon):
    #Calculate great circle distance. Using geopy.great_circle
    dists = []
    for i in range(n_files):
        dists.append(great_circle((tonga_latlon[0], tonga_latlon[1]) , (lats[i], lons[i])).m)

    dists = np.array(dists)
    dists_km = dists/1000

    print('Smallest great cricle distance is {:.2f}km. Largest great circle distance is {:.2f}km'.format(
        np.min(dists_km), np.max(dists_km)))

    plt.style.use('seaborn-v0_8-whitegrid')
    n_vec = np.linspace(0,n_files-1,n_files)
    fig = plt.figure(figsize=(15,10))
    ax = fig.add_subplot(111)
    ax.scatter(n_vec, np.sort(dists_km), s=0.5)
    ax.set_xlabel('Station number', fontsize=28)
    ax.set_ylabel('Distance, [km]', fontsize=28)
    ax.set_title('Great circle distances between Hunga Tonga and stations', fontsize=28)
    plt.show()

    return dists_km

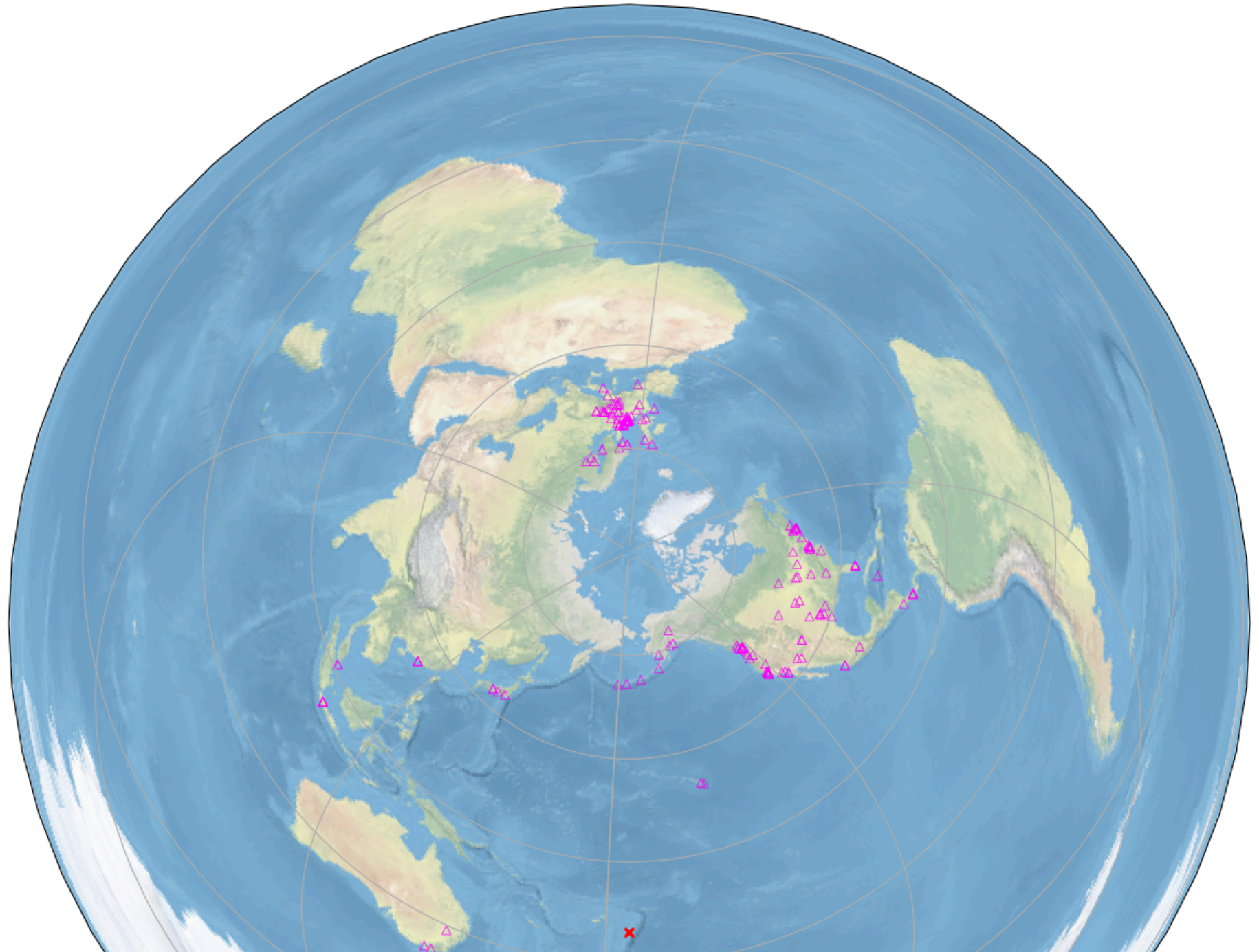
# READ DATA
data_collection, times_collection, lats, lons, dt = parse()
n_files = len(lats)

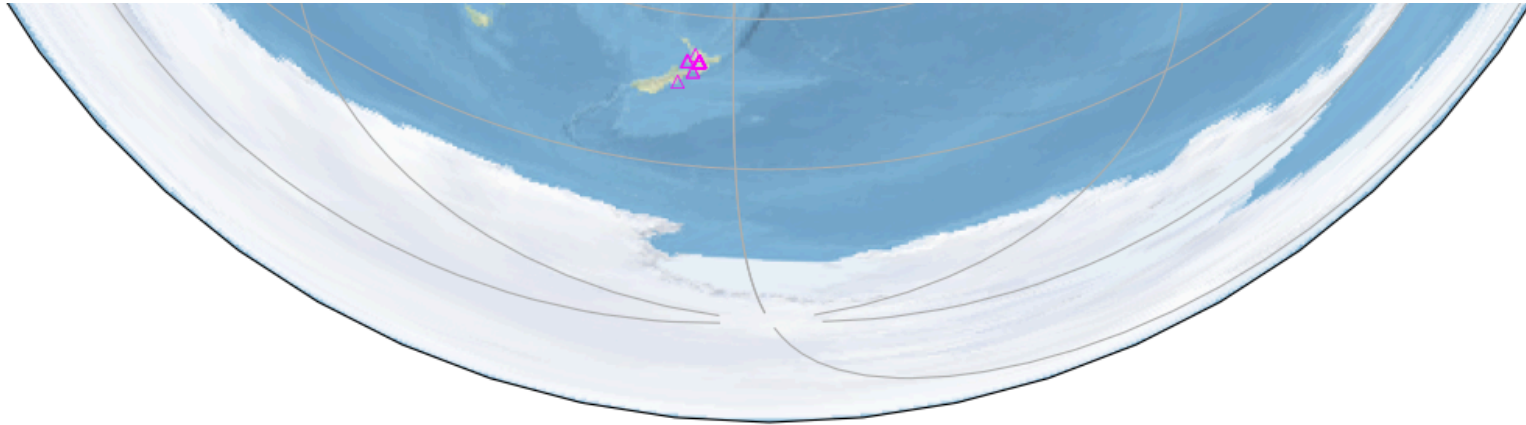
# Hunga Tonga location
tonga_latlon = [-20.550, -175.385] # latitude and longitude

```

```
# Map  
plot_map(lats, lons, tonga_latlon)  
  
dists_km = circle_distance(n_files, lats, lons, tonga_latlon)
```

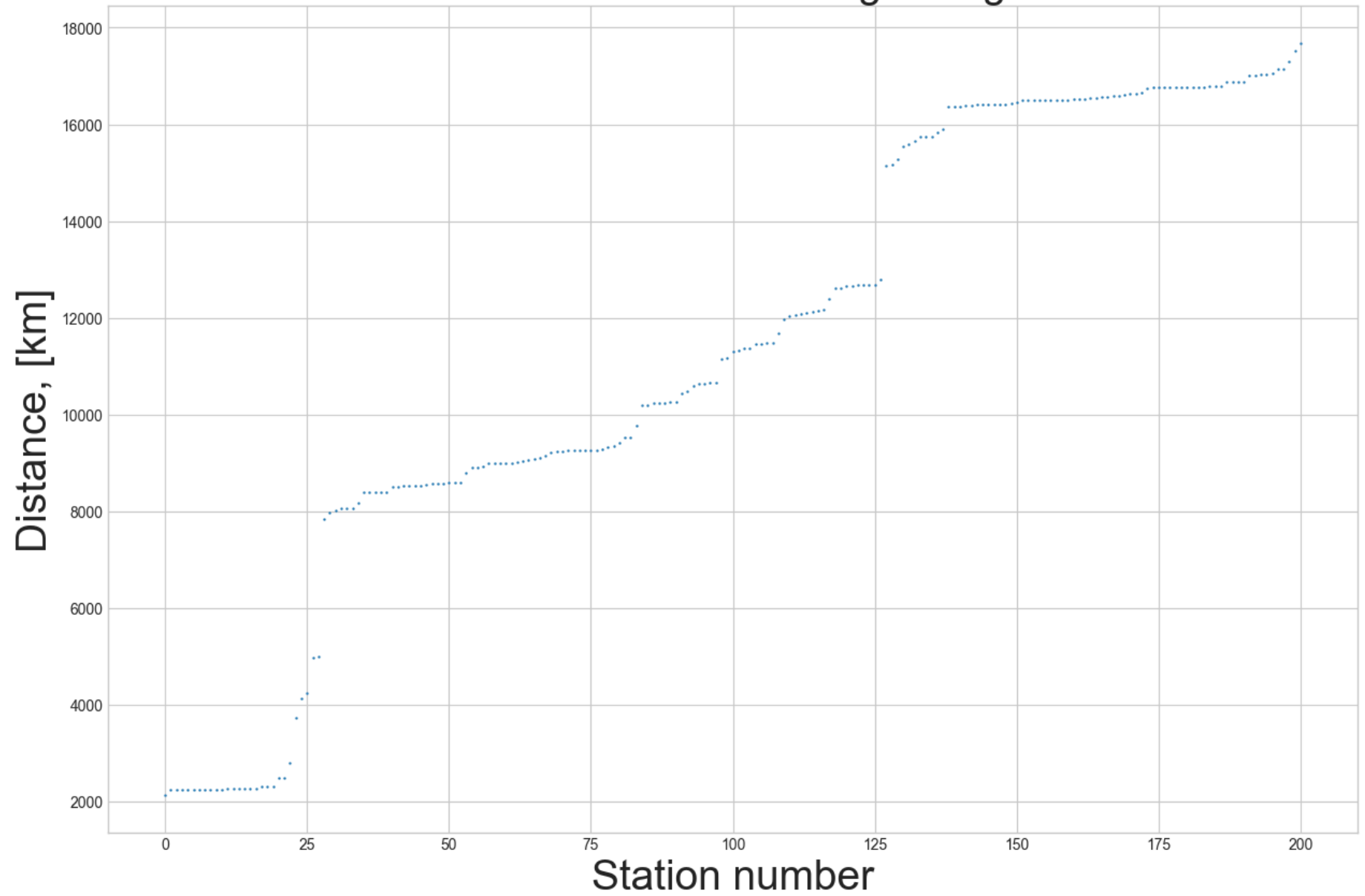
```
Loading station 0 out of 200  
Loading station 50 out of 200  
Loading station 100 out of 200  
Loading station 150 out of 200  
Loading station 200 out of 200  
done
```





Smallest great cricle distance is 2137.71km. Largest great circle distance is 17687.13km

Great circle distances between Hunga Tonga and stations



The provided script was adapted to Python to load the Hunga Tonga data. The station locations were shown on a map, and the distances between Hunga Tonga and each station were calculated and plotted in order. The shortest distance was **2137.71 km**, and the longest was **17687.13 km**.

Task 2

a)

```
In [2]: # Loop that turns impulse textfile into lists in matlab format to Python lists
with open("impulse_response.txt") as file:
    text = file.read()

lists = []

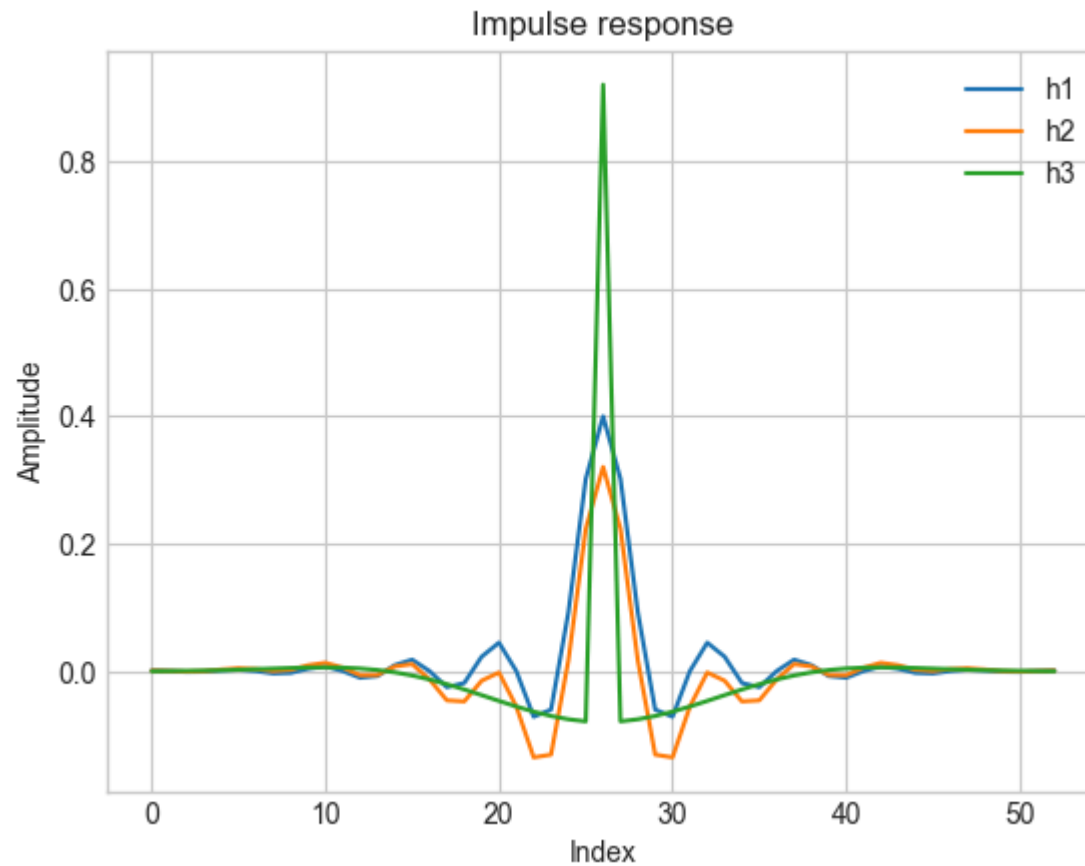
for section in text.split(";"):
    if "[" not in section:
        continue

    numbers = section.split("[")[1].split(" ")[0]
    number_list = [float(n) for n in numbers.split()]
    lists.append(number_list)

h1, h2, h3 = lists

x_verdier = np.arange(0, len(h1))

plt.plot(x_verdier, h1, label = "h1")
plt.plot(x_verdier, h2, label = "h2")
plt.plot(x_verdier, h3, label = "h3")
plt.xlabel("Index")
plt.ylabel("Amplitude")
plt.title("Impulse response")
plt.legend()
plt.show()
```



The three FIR filter impulse responses, $(h_1[n])$, $(h_2[n])$, and $(h_3[n])$, were loaded from the provided text file and converted into Python lists. The impulse responses were then plotted together to compare their amplitudes at each sample index.

b)

```
In [3]: def convolve(input_signal, impulse_response, ylen_choice):
        x = np.array(input_signal)
        h = np.array(impulse_response)

        if ylen_choice == 0:
            y = np.zeros(len(x))
```

```

    for n in range(len(x)):
        for k in range(len(x)):
            if 0 <= n-k < len(h):
                y[n] += x[k] * h[n-k]

    return y

else:
    y = np.zeros((len(x) + len(h)) - 1)

    for n in range(len(y)):
        for k in range(len(x)):
            if 0 <= n-k < len(h):
                y[n] += x[k] * h[n-k]

    return y

# Testing if the function works
x = np.array([1, 2, 3])
h = np.array([1, 2])

print(convolve(x, h, 1))
print(np.convolve(x, h, mode="full"))

```

```
[1.  4.  7.  6.]
```

```
[1  4  7  6]
```

A convolution function was implemented using two `for` loops to calculate the output signal ($y[n]$). The function takes the input signal ($x[n]$), impulse response ($h[n]$), and `ylen_choice` as inputs. When `ylen_choice = 0`, the output has the same length as the input signal. When `ylen_choice = 1`, the full convolution is calculated with a length of $(M+N-1)$.

c)

```

In [4]: def DTFT(input_signal, N, fs):
        h = np.array(input_signal)
        y = np.zeros(N, dtype = complex)
        length = len(h)

```

```

freq = np.array([(k*fs)/N for k in range(0, N)])
omega = 2 * np.pi * freq / fs

for k in range(0, len(y)):
    for n in range(0, length):
        y[k] += h[n] * np.exp(-1j * omega[k] * n)

return y, freq

# Test of DTFT-function
h = np.array([1, 2, 1])
H, freq = DTFT(h, 4, 1000)

print("Frequency (Hz):", freq)
print("DTFT:", np.round(H, 4))

```

Frequency (Hz): [0. 250. 500. 750.]

DTFT: [4.+0.j 0.-2.j 0.+0.j -0.+2.j]

A DTFT function was implemented using two `for` loops to calculate the frequency spectrum of an input signal. The function takes the signal, the number of frequency points `N`, and the sampling frequency `fs` as inputs. It returns the complex frequency spectrum and the corresponding frequency axis in Hertz. The function was tested using the signal `[1, 2, 1]` with `N = 4` and `fs = 1000`.

d)

```

In [5]: N = 1000
        fs = 1000

        # Calculate DTFT
        H1, freq = DTFT(h1, N, fs)
        H2, freq2 = DTFT(h2, N, fs)
        H3, freq3 = DTFT(h3, N, fs)

        # Plots
        fig, axs = plt.subplots(3, 1, figsize=(10, 10), sharex=True)

        # H1
        axs[0].plot(freq, np.abs(H1), color="blue")
        axs[0].set_title("Frequency Response H1")

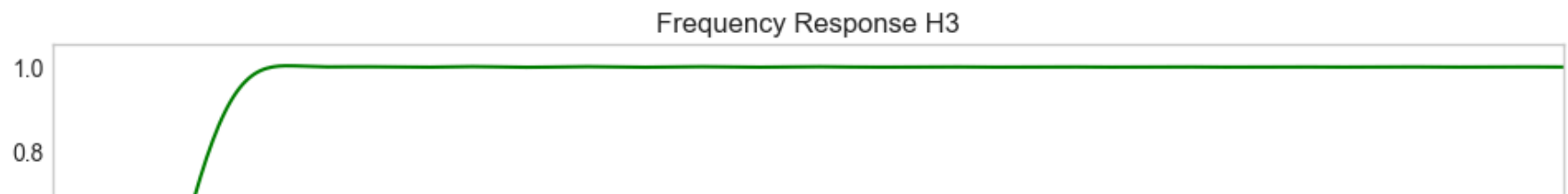
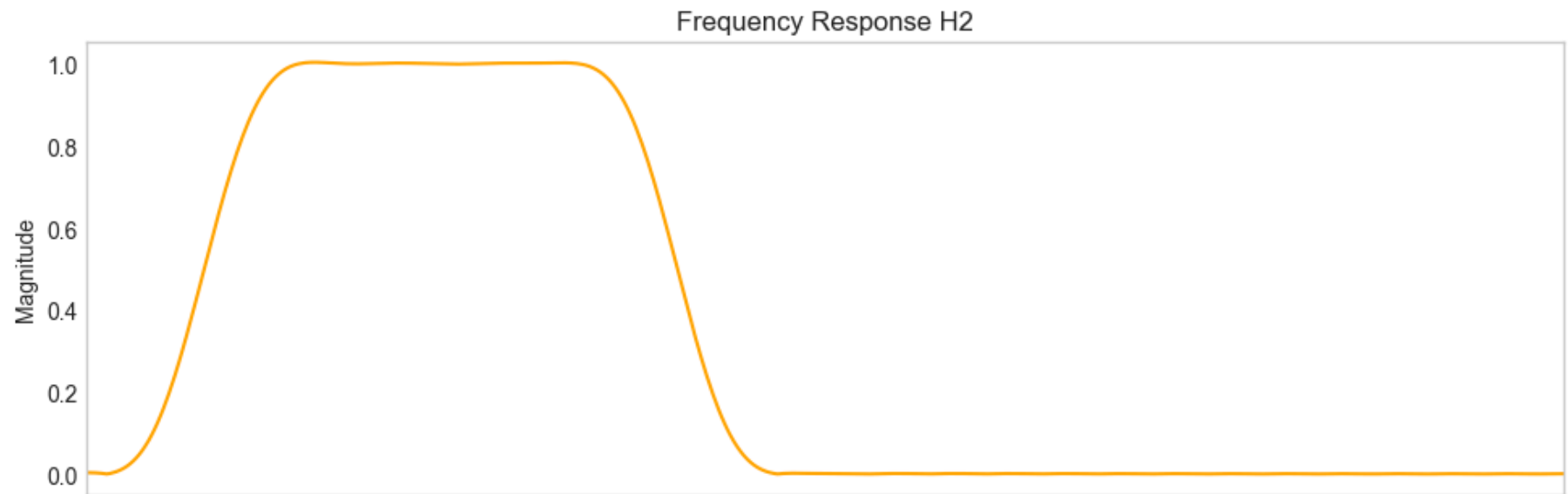
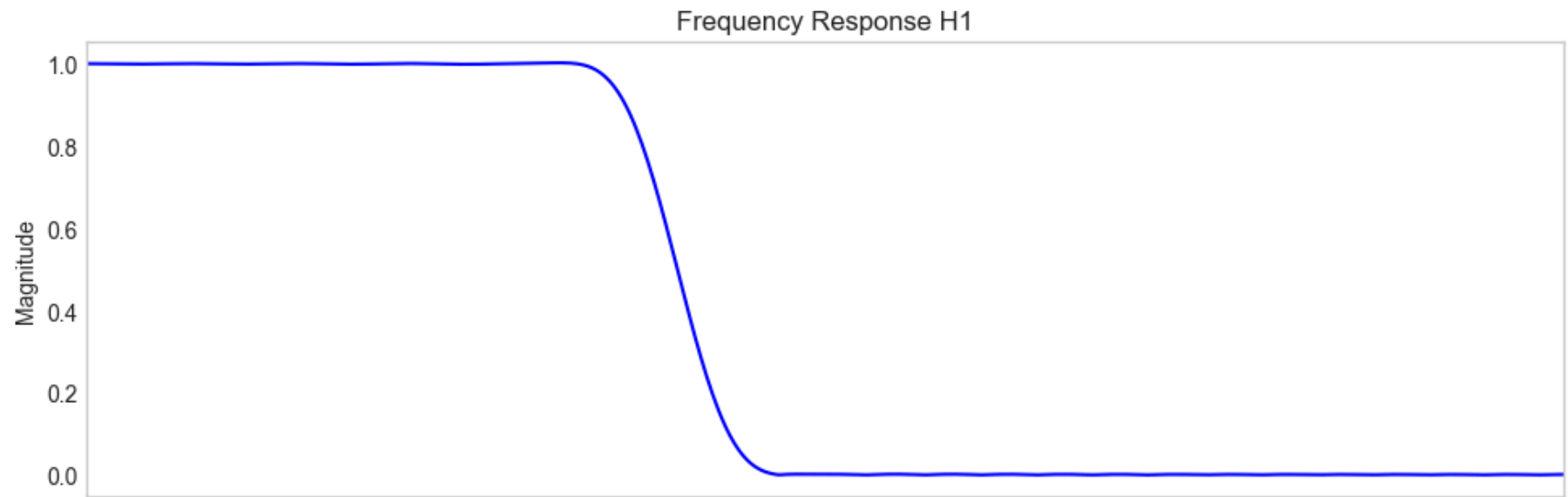
```

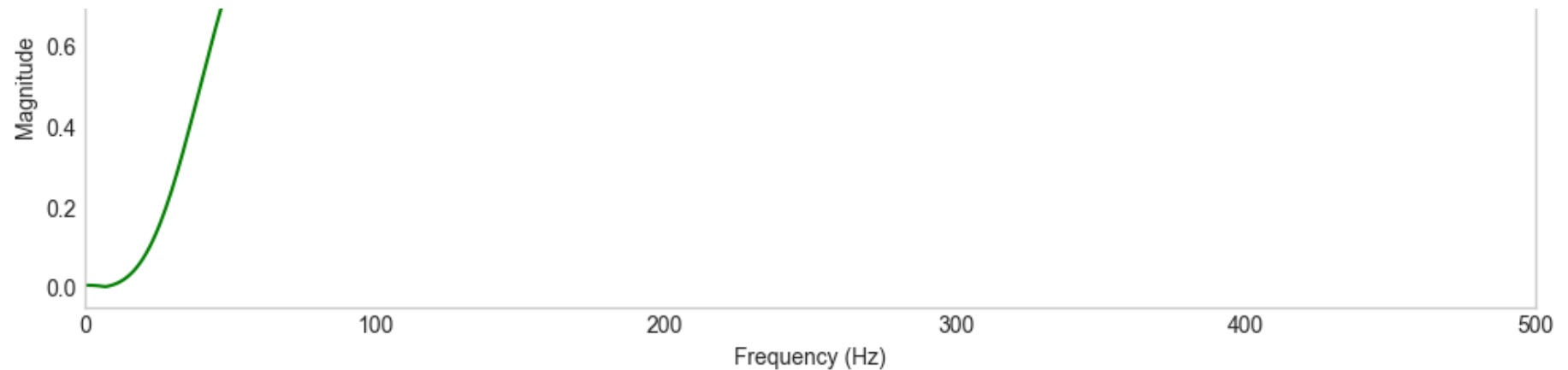
```
axs[0].set_ylabel("Magnitude")
axs[0].grid()

# H2
axs[1].plot(freq, np.abs(H2), color="orange")
axs[1].set_title("Frequency Response H2")
axs[1].set_ylabel("Magnitude")
axs[1].grid()

# H3
axs[2].plot(freq, np.abs(H3), color="green")
axs[2].set_title("Frequency Response H3")
axs[2].set_ylabel("Magnitude")
axs[2].set_xlabel("Frequency (Hz)")
axs[2].set_xlim(0, fs / 2)
axs[2].grid()

plt.tight_layout()
plt.show()
```





The DTFT function was used to calculate the frequency responses of the three impulse responses, using `N = 1000` and `fs = 1000` Hz. The absolute values were calculated using `np.abs()` and plotted separately against frequency in Hertz. The plots show the frequency range from 0 to 500 Hz, which is the Nyquist frequency.

e)

Based on the frequency responses, the three filters can be classified as follows:

h1[n] is a low-pass filter, allowing low frequencies to pass while reducing higher frequencies.

h2[n] is a band-pass filter, allowing a middle range of frequencies to pass while reducing lower and higher frequencies.

h3[n] is a high-pass filter, allowing high frequencies to pass while reducing lower frequencies.

f)

```
In [6]: # Store the filtered signals
        filtered_h1 = []
        filtered_h2 = []
        filtered_h3 = []

        for x in data_collection:
```

```

    filtered_h1.append(np.convolve(x, h1, mode="same"))
    filtered_h2.append(np.convolve(x, h2, mode="same"))
    filtered_h3.append(np.convolve(x, h3, mode="same"))

# Convert to Numpy arrays
    filtered_h1 = np.array(filtered_h1)
    filtered_h2 = np.array(filtered_h2)
    filtered_h3 = np.array(filtered_h3)

    print("Original data:", data_collection.shape)
    print("Low-pass:", filtered_h1.shape)
    print("Band-pass:", filtered_h2.shape)
    print("High-pass:", filtered_h3.shape)

```

Original data: (201, 720000)

Low-pass: (201, 720000)

Band-pass: (201, 720000)

High-pass: (201, 720000)

All 201 input time series were filtered using the three FIR filters. NumPy's built-in `np.convolve()` function was used with `mode="same"` to keep the output length the same as the input signal. The filtered signals were stored in three separate NumPy arrays, one for each filter. The array dimensions were then checked to confirm that all 201 signals had been processed.

g)

```

In [7]: plt.figure(figsize=(12, 8))
# Common reference time
    start = times_collection[0, 0]
    for i in range(len(dists_km)):

        # Band-pass filtered signal
        signal = filtered_h2[i]

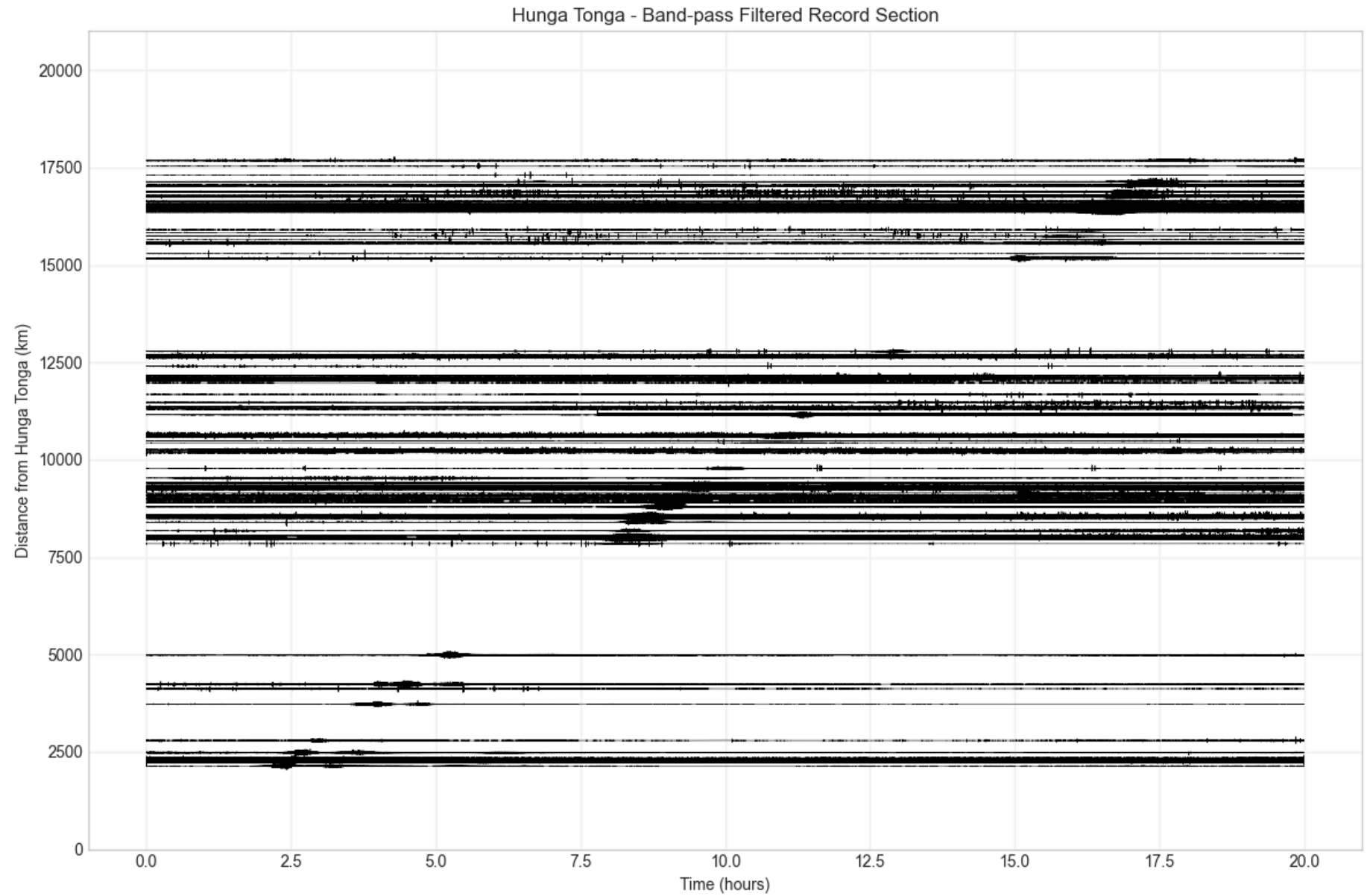
        # Time in hours relative to the common reference
        offset = (times_collection[i, 0] - start).total_seconds() / 3600
        time = offset + np.arange(len(signal)) * dt[i] / 3600

        # Normalize the signal
        signal = signal / (np.max(np.abs(signal)) + 1e-12)

```

```
# Plot every 20th sample
plt.plot(time,
         dists_km[i] + signal * 100,
         color="black", linewidth=0.4)

plt.xlabel("Time (hours)")
plt.ylabel("Distance from Hunga Tonga (km)")
plt.title("Hunga Tonga - Band-pass Filtered Record Section")
plt.ylim(0, 21000)
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```



The band-pass filter ($h_2[n]$) was chosen because it reduces frequencies outside an intermediate frequency range, similar to the filter used by Matoza et al. (2022).

The filtered signals from all 201 stations were plotted with time on the x-axis and distance from Hunga Tonga on the y-axis. Each signal was normalized and positioned according to its station's distance. Some wave arrivals are visible, but the propagation pattern is difficult to see clearly because many of the signals overlap.

Task 3

a)

```
In [ ]: def pick_arrivals(start_index, stop_index, session_name):

    # Sort stations by distance from Hunga Tonga
    order = np.argsort(dists_km)

    # Store results
    station_indices = []
    arrival_times = []

    # Common reference time
    start = times_collection[0, 0]

    for i in order[start_index:stop_index]:

        # Time axis in hours
        offset = (times_collection[i, 0] - start).total_seconds() / 3600

        # Plot every 20th sample
        idx = np.arange(0, len(data_collection[i]), 20)
        time = offset + idx * dt[i] / 3600

        # Create four subplots
        fig, axs = plt.subplots(4, 1, figsize=(12, 8), sharex=True)

        axs[0].plot(time, data_collection[i, idx])
        axs[1].plot(time, filtered_h1[i, idx])
        axs[2].plot(time, filtered_h2[i, idx])
        axs[3].plot(time, filtered_h3[i, idx])
```

```

# Titles
axs[0].set_title("Raw signal")
axs[1].set_title("Low-pass")
axs[2].set_title("Band-pass")
axs[3].set_title("High-pass")

for ax in axs:
    ax.set_ylabel("Amplitude")
    ax.grid()

axs[3].set_xlabel("Time (hours)")

fig.suptitle(f"Station {i} - Distance: {dists_km[i]:.0f} km")
plt.tight_layout()
plt.show(block=False)

# Click on the high-frequency arrival
print(f"Station {i}: Click on the high-frequency wave arrival")

point = plt.ginput(1, timeout=0)

if point:
    arrival = point[0][0]
else:
    arrival = np.nan

# Store the result
station_indices.append(i)
arrival_times.append(arrival)

# Close the figure
plt.close(fig)

# Save results after each station
np.savez(
    f"picks_{session_name}.npz",
    station_indices=np.array(station_indices),
    arrival_times=np.array(arrival_times),
    reference_time=start.isoformat()
)

```

```
return np.array(station_indices), np.array(arrival_times)
```

```
In [ ]: %matplotlib osx

# Session 1: Stations 0-49
stations1, arrivals1 = pick_arrivals(0, 50, "session1")

# Session 2: Stations 50-99
stations2, arrivals2 = pick_arrivals(50, 100, "session2")

# Session 3: Stations 100-149
stations3, arrivals3 = pick_arrivals(100, 150, "session3")

# Session 4: Stations 150-200
stations4, arrivals4 = pick_arrivals(150, 201, "session4")

%matplotlib inline
```

A function was created to manually select the arrival times of the high-frequency infrasound waves. The stations were sorted by distance from Hunga Tonga, and four plots were displayed for each station: the original signal and the three filtered signals.

The arrival times were selected by clicking on the plots using `plt.ginput()`. The results were saved together with their station indices in four separate `.npz` files, allowing the analysis to be completed in multiple sessions.

b)

```
In [8]: # Load saved arrival times
stations = []
arrivals = []

for session in range(1, 5):
    data = np.load(f"picks_session{session}.npz")

    stations.extend(data["station_indices"])
    arrivals.extend(data["arrival_times"])
```

```
stations = np.array(stations)
arrivals = np.array(arrivals)

# Get the distance for each station
distance = dists_km[stations]

print("Number of stations:", len(stations))
```

Number of stations: 201

The saved arrival times from the four picking sessions were loaded and combined into NumPy arrays. Each arrival time was matched with its station's distance from Hunga Tonga for further analysis.

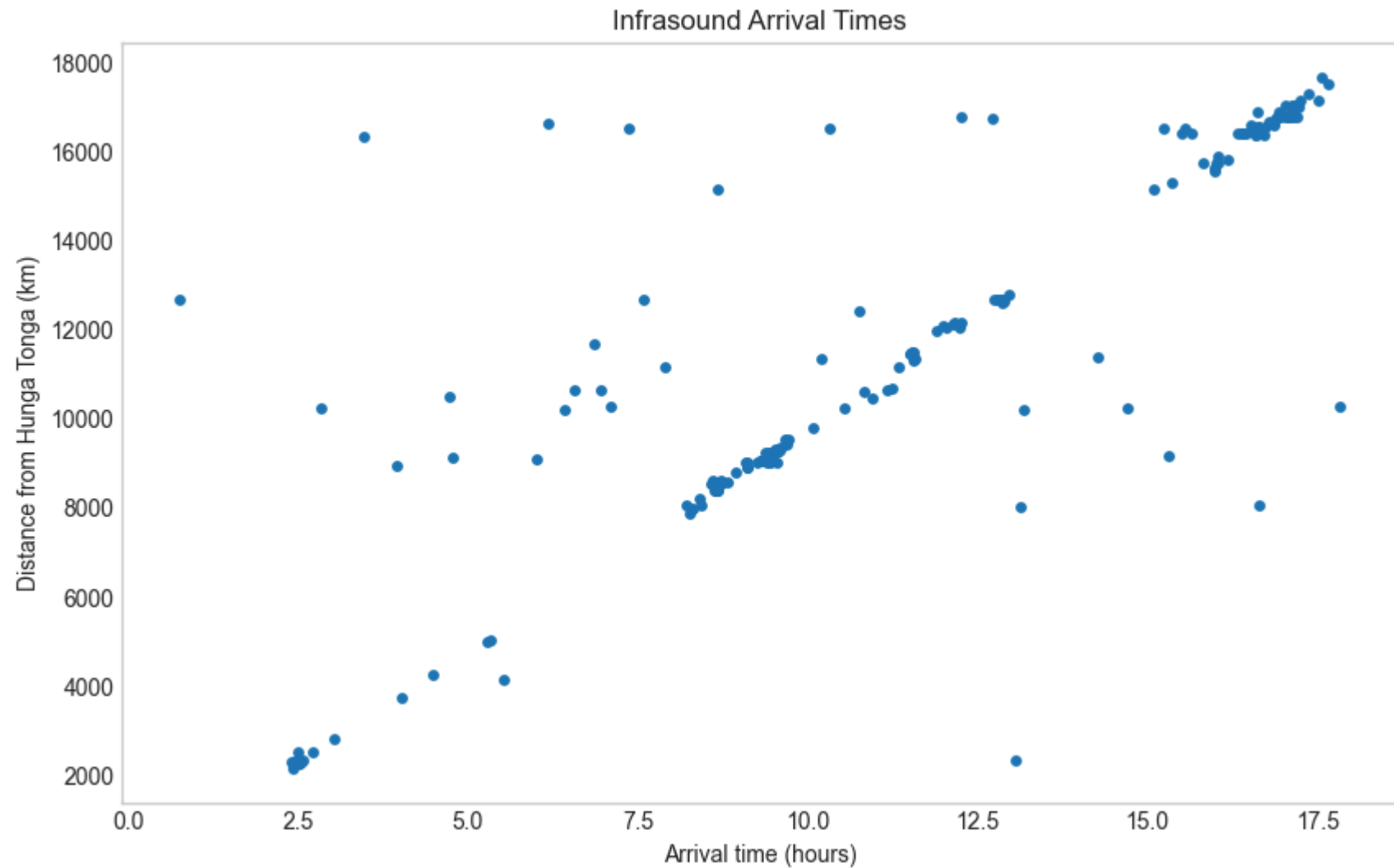
```
In [9]: %matplotlib inline

plt.figure(figsize=(10, 6))

plt.scatter(arrivals, distance, s=15)

plt.xlabel("Arrival time (hours)")
plt.ylabel("Distance from Hunga Tonga (km)")
plt.title("Infrasound Arrival Times")

plt.grid()
plt.show()
```



The selected arrival times were plotted against the distances from Hunga Tonga. The plot shows the relationship between arrival time and distance and helps identify possible outliers.

```
In [10]: %matplotlib inline  
  
# Convert to seconds and meters  
time_s = arrivals * 3600
```

```

distance_m = distance * 1000

# Initial linear fit
c, b = np.polyfit(time_s, distance_m, 1)

# Calculate residuals and standard deviation
residuals = distance_m - (c * time_s + b)
std = np.std(residuals)

# Create two plots side by side
fig, axs = plt.subplots(1, 2, figsize=(15, 6), sharex=True, sharey=True)

# Compare 1 and 2 standard deviations
for ax, n_std in zip(axs, [1, 2]):

    # Identify outliers
    valid = np.abs(residuals) <= n_std * std

    # Fit without outliers
    c, b = np.polyfit(time_s[valid], distance_m[valid], 1)

    # Plot valid picks
    ax.scatter(arrivals[valid], distance[valid],
               s=15, label="Valid picks")

    # Plot outliers
    ax.scatter(arrivals[~valid], distance[~valid],
               color="red", marker="x", label="Outliers")

    # Plot fitted line
    t = np.linspace(time_s[valid].min(), time_s[valid].max(), 100)

    ax.plot(t / 3600, (c * t + b) / 1000,
            color="black", label="Linear fit")

    # Titles and labels
    ax.set_title(f"{n_std} Standard Deviation\nCelerity: {c:.2f} m/s")
    ax.set_xlabel("Arrival time (hours)")
    ax.grid()
    ax.legend()

```

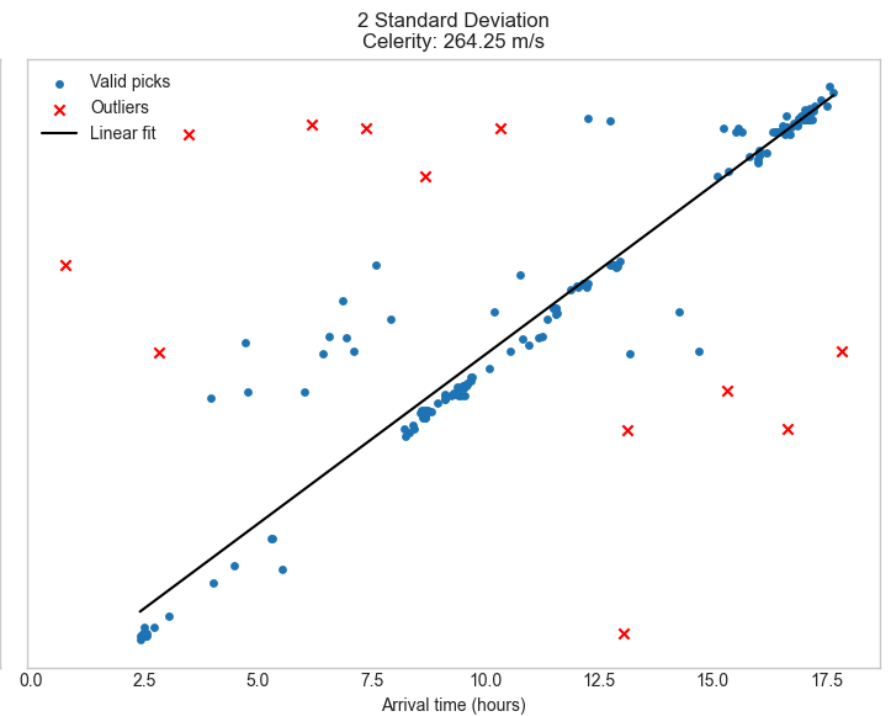
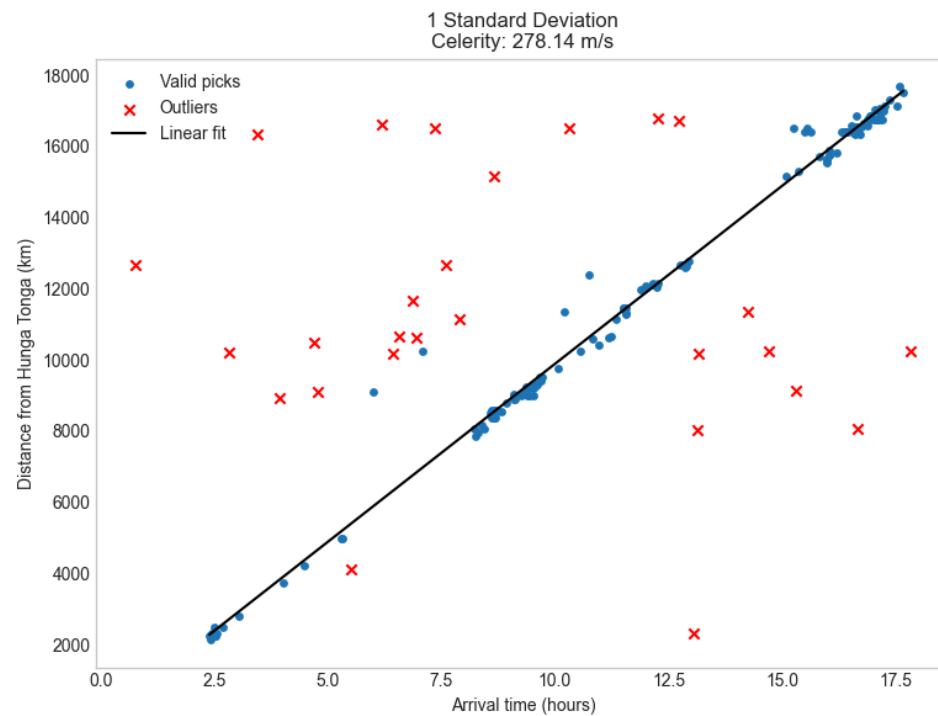
```
# Print results
print(f"{n_std}σ: {c:.2f} m/s")
print(f"Valid stations: {np.sum(valid)}")
print(f"Outliers: {np.sum(~valid)}\n")

axs[0].set_ylabel("Distance from Hunga Tonga (km)")

plt.tight_layout()
plt.show()
```

1σ: 278.14 m/s
Valid stations: 174
Outliers: 27

2σ: 264.25 m/s
Valid stations: 189
Outliers: 12



The arrival times were plotted against the distances from Hunga Tonga. A linear fit was used to estimate the celerity, and possible outliers were identified using one and two standard deviations from the initial fit. The outliers are shown as red crosses and were excluded from the final calculations.

Using one standard deviation, 27 outliers were removed, giving an estimated celerity of **278.14 m/s**. Using two standard deviations, 12 outliers were removed, giving an estimate of **264.25 m/s**.

Both results are within the typical ranges for atmospheric infrasound reported by Vergoz et al. (2022, p. 3). The difference between the estimates shows that the calculated celerity depends on which data points are excluded.

Conclusion

In this project, convolution and DTFT functions were implemented, and three FIR filters were identified as low-pass, band-pass, and high-pass filters. The filters were applied to 201 recorded time series from the Hunga Tonga eruption.

The arrival times of the high-frequency infrasound waves were manually selected and used to estimate their celerity. Linear regression gave estimates of **278.14 m/s** and **264.25 m/s**, depending on the outlier threshold. Both estimates fall within typical ranges for atmospheric infrasound reported by Vergoz et al. (2022).

The results show how digital signal processing and linear regression can be used to study the propagation of atmospheric waves.